

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development	Experiment No.	Task 7
Course Code	23CM4121	Student Name	R Abhiram
Reg. No.	A24126552114	Date	21-09-2026

1. TITLE

Document Object Model (DOM) and Dynamic Manipulation of Content, Style and Attributes of an HTML Element Using JavaScript

2. AIM

To explain the **Document Object Model (DOM)** and to demonstrate, through a working web page, how JavaScript can dynamically change the **content**, **attributes** and **style** of an HTML element when the user clicks a button.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the DOM as a tree-structured, in-memory representation of an HTML document.
- To select HTML elements from JavaScript using `document.getElementById()`.
- To change the text content of an element using the `textContent` property.
- To change element attributes using `setAttribute()` (src and alt of an image).
- To change appearance using `classList.add()` and the `inline style` property.
- To respond to user actions using `addEventListener()` and the `click` event.
- To verify the before and after state of the page through screenshots and DOM inspection.

4. THEORY

The **Document Object Model (DOM)** is a programming interface for HTML and XML documents. When a browser loads a web page, it parses the HTML and builds a **tree of objects** in memory, where every tag, attribute and piece of text becomes a **node**. JavaScript can read and modify this tree, and the browser immediately re-renders the page to reflect the change. In this way a static HTML page becomes dynamic and interactive without reloading.

- **document:** the root object of the DOM; the entry point for accessing every element on the page.
- **Element node:** represents an HTML tag such as `<div>`, `<h2>` or `<button>`.
- **Attribute node:** represents an attribute of an element such as `id`, `class`, `src` or `alt`.
- **Text node:** the text inside an element, for example "Default Title" inside the `<h2>`.
- **Tree relationships:** nodes have parent, child and sibling relationships, e.g. `<body>` is the parent of `<div id="card">`.

DOM tree of the page used in this experiment

```
document
  |-- html
    |-- head
      |-- style
    |-- body
      |-- div#card.card
        |-- h2#card-title    -> text: "Default Title"
        |-- img#card-image  -> src, alt
        |-- br
        |-- br
        |-- button#update-btn -> click event listener
      |-- script
```

Ways JavaScript changes an element through the DOM

What is changed	JavaScript used	Effect in this program
Content	<code>element.textContent = "..."</code>	Heading text changes from "Default Title" to "Profile Loaded".
Attributes	<code>element.setAttribute(name, value)</code>	The image src and alt attributes are replaced with new values.
Style (CSS class)	<code>element.classList.add("highlight")</code>	The card receives the highlight class: blue border and light-blue background.
Style (inline)	<code>element.style.color = "#1e40af"</code>	The heading text colour changes to dark blue.

textContent vs innerHTML: textContent inserts plain text and is safe from HTML injection, whereas innerHTML parses the string as HTML. **classList vs style:** classList lets the stylesheet decide how the element looks (preferred for reusable styling), while style sets values directly on the element and overrides the stylesheet. **Events:** `addEventListener("click", fn)` registers a function that the browser runs every time the element is clicked.

Flow: page loads and DOM is built → button clicked → JavaScript selects the card, title and image → content, attributes, class and inline style are updated → browser re-renders the updated card.

5. ALGORITHM / PROCEDURE

- 1 Create an HTML file with a `<div id="card">` containing an `<h2>`, an `<img>` and a `<button>`.
- 2 Define the CSS classes `.card` (default look) and `.highlight` (blue border and background).
- 3 In a `<script>` block, select the button with `document.getElementById("update-btn")`.
- 4 Attach a click event listener to the button using `addEventListener()`.
- 5 Inside the listener, select the card, the title and the image elements by their ids.
- 6 **Change content:** set `title.textContent` to "Profile Loaded".
- 7 **Change attributes:** use `setAttribute()` to update the src and alt of the image.
- 8 **Change style:** add the highlight class to the card with `classList.add()` and set `title.style.color`.
- 9 Open the file in a browser, observe the default card, click "Update Card" and observe the updated card.
- 10 Verify the DOM values before and after the click and repeat the click to check the behaviour.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <style>
    .card {
      border: 1px solid #ccc;
      padding: 16px;
      border-radius: 8px;
      width: 250px;
      font-family: sans-serif;
    }
    .highlight {
      border-color: #2563eb;
      background-color: #eff6ff;
    }
  </style>
</head>
<body>

  <div id="card" class="card">
    <h2 id="card-title">Default Title</h2>
    
    <br><br>
    <button id="update-btn">Update Card</button>
  </div>

  <script>
    const button = document.getElementById('update-btn');

    button.addEventListener('click', () => {
      const card = document.getElementById('card');
      const title = document.getElementById('card-title');
      const image = document.getElementById('card-image');

      // 1. Change Content
      title.textContent = "Profile Loaded";

      // 2. Change Attributes
      image.setAttribute('src', 'https://via.placeholder.com/100/2563eb/ffffff');
      image.setAttribute('alt', 'Updated Profile Picture');

      // 3. Change Style & Classes
      card.classList.add('highlight');
      title.style.color = '#1e40af';
    });
  </script>
</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<code>.card { ... }</code>	Default style of the card: grey 1px border, 16px padding, rounded corners, 250px width, sans-serif font.
<code>.highlight { ... }</code>	Style applied after the click: blue border colour and light-blue background colour.
<code><div id="card" class="card"></code>	Container element that acts as the profile card; it is selected later by its id.
<code><h2 id="card-title"></code>	Heading whose text and colour are changed by JavaScript.
<code></code>	Image whose src and alt attributes are changed by JavaScript.
<code>document.getElementById('update-button')</code>	Returns the DOM element object of the button so that it can be used in JavaScript.
<code>button.addEventListener('click', ...)</code>	Registers an arrow function that runs every time the button is clicked.
<code>title.textContent = "Profile Loaded";</code>	Changes the content of the heading.
<code>image.setAttribute('src', ...)</code>	Changes the attribute src; the image now points to a different URL.
<code>image.setAttribute('alt', ...)</code>	Changes the attribute alt (alternative text shown or read when the image is unavailable).
<code>card.classList.add('highlight');</code>	Adds a CSS class to the card, so the highlight rules apply. Adding a class that is already present does nothing.
<code>title.style.color = '#1e40af';</code>	Changes the style directly by setting an inline colour on the heading.

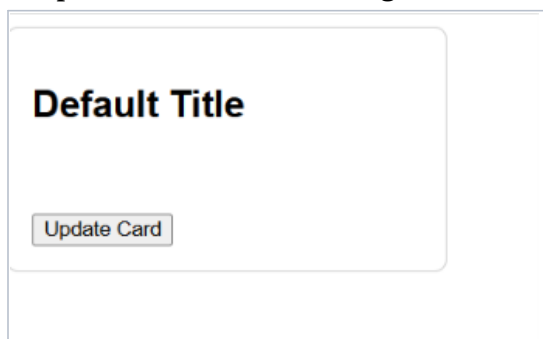
8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and each test case was carried out as described below. Each test was run in a browser; a screenshot was taken and the DOM values were read from the live page and compared with the expected results (see Section 9).

Test Case	Action	Property Checked	Expected Result	Status
TC-01	Load page, no click	Title text, card class, border and background	"Default Title"; class "card"; grey border; no background	Pass
TC-02	Click "Update Card" once	Content: textContent of h2	Text becomes "Profile Loaded"	Pass
TC-03	Click "Update Card" once	Attributes: src and alt of img	src ends with /100/2563eb/ffffff; alt = "Updated Profile Picture"	Pass
TC-04	Click "Update Card" once	Style: class list, border and background of card	class "card highlight"; blue border; light-blue background	Pass
TC-05	Click "Update Card" once	Style: inline colour of h2	style attribute contains colour #1e40af (dark blue)	Pass
TC-06	Click "Update Card" three times in total	Repeat clicks: duplicates and extra nodes	No duplicate class, values unchanged, node count still 11	Pass

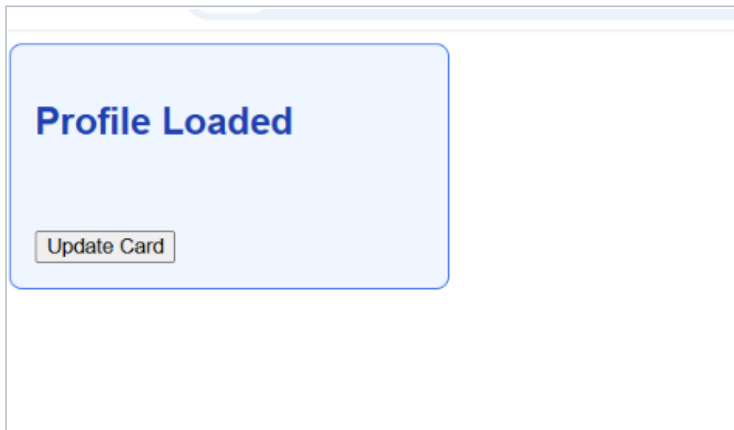
9. OUTPUT

Output 1: Card before clicking the button (default state)



The card shows the default heading "Default Title" with a grey border and no background colour.

Output 2: Card after clicking "Update Card"



The heading now reads "Profile Loaded" in dark blue, and the card has a blue border and light-blue background. The image area is not visible in the screenshots because the external placeholder image URL did not render in the browser; the change to its src and alt attributes is shown in the test case outputs below.

Test case outputs (screenshots)

Output 3: TC-01 – Page loaded, before click

TC-01 – Page loaded (no click) PASS

Before click

Default Title



Update Card

Property checked	Expected	Actual (read from page)	Result
h2.textContent	"Default Title"	"Default Title"	PASS
card.className	"card"	"card"	PASS
card border colour	rgb(204, 204, 204)	rgb(204, 204, 204)	PASS
card background	transparent	rgba(0, 0, 0, 0)	PASS

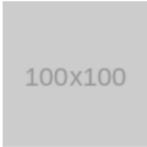
Screenshot of the page on load, with the DOM values read from it. The default state matches Output 1.

Output 4: TC-02 – Content changed

TC-02 – Content changedPASS

Before click

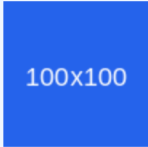
Default Title



Update Card

After click

Profile Loaded



Update Card

Property checked	Expected	Actual (read from page)	Result
h2.textContent (before)	"Default Title"	"Default Title"	PASS
h2.textContent (after)	"Profile Loaded"	"Profile Loaded"	PASS

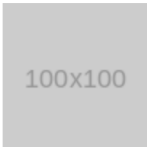
The heading text changed from "Default Title" to "Profile Loaded" after the click.

Output 5: TC-03 – Attributes changed

TC-03 – Attributes changedPASS

Before click

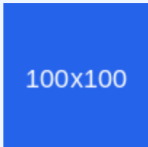
Default Title



Update Card

After click

Profile Loaded



Update Card

Property checked	Expected	Actual (read from page)	Result
img.src (before)	.../100	.../100	PASS
img.src (after)	.../100/2563eb/ffffff	.../100/2563eb/ffffff	PASS
img.alt (before)	"Placeholder"	"Placeholder"	PASS
img.alt (after)	"Updated Profile Picture"	"Updated Profile Picture"	PASS

The src and alt attributes of the image changed after the click (image URLs served by a local stand-in of the same colours so the change is visible).

Output 6: TC-04 – Class and card style changed

TC-04 – Class and card style changedPASS

Before click

Default Title

100x100

Update Card

After click

Profile Loaded

100x100

Update Card

Property checked	Expected	Actual (read from page)	Result
card classList (after)	['card', 'highlight']	['card', 'highlight']	PASS
card border colour (after)	rgb(37, 99, 235)	rgb(37, 99, 235)	PASS
card background (after)	rgb(239, 246, 255)	rgb(239, 246, 255)	PASS

The highlight class was added; the card now has a blue border and a light-blue background.

Output 7: TC-05 – Inline style changed

TC-05 – Inline style changedPASS

Before click

Default Title

100x100

Update Card

After click

Profile Loaded

100x100

Update Card

Property checked	Expected	Actual (read from page)	Result
h2 style attribute (before)	none	null	PASS
h2 style attribute (after)	colour set	color: rgb(30, 64, 175);	PASS
h2 computed colour (after)	rgb(30, 64, 175)	rgb(30, 64, 175)	PASS

title.style.color added an inline style; the heading colour changed from black to dark blue.

Output 8: TC-06 – Button clicked three times

TC-06 – Button clicked three times

PASS

After 1 click

Profile Loaded

100x100

Update Card

After 3 clicks

Profile Loaded

100x100

Update Card

Property checked	Expected	Actual (read from page)	Result
card className	"card highlight"	"card highlight"	PASS
h2.textContent	"Profile Loaded"	"Profile Loaded"	PASS
img alt	"Updated Profile Picture"	"Updated Profile Picture"	PASS
elements in page	11	11	PASS

Repeated clicks leave the page in the same state as a single click: no duplicate class and no extra elements.

10. RESULT

Thus, the Document Object Model was studied and a web page was developed in which JavaScript dynamically changed the **content** (textContent), **attributes** (setAttribute) and **style** (classList and style.color) of HTML elements when a button was clicked. The before and after screenshots and the DOM inspection results of six test cases confirmed that the page behaved as expected.